

NanoFabric

Build a Tiny Nutanix to Get Hired by Nutanix

DISTRIBUTED SYSTEMS	SELF-HEALING STORAGE	NUTANIX-ALIGNED
---------------------	----------------------	-----------------

Language	Python 3.11+
Framework	FastAPI + Docker
Duration	4 Weeks (~1–2 hrs/day)
Difficulty	Intermediate → Advanced
Target Role	MTS / SRE / Support / QA at Nutanix

Table of Contents

01	Introduction & Project Vision
02	Prerequisites & Preparation
03	Tech Stack Deep Dive
04	Architecture Overview
05	4-Week Roadmap
06	Module-by-Module Build Guide
06.1	Module 0 — Project Scaffolding
06.2	Module 1 — Local Storage Engine
06.3	Module 2 — Cluster Membership & Heartbeats
06.4	Module 3 — Distributed Metadata
06.5	Module 4 — Replication Factor & Life of a Write
06.6	Module 5 — Quorum & Split-Brain Prevention
06.7	Module 6 — Self-Healing / Re-Protection
06.8	Module 7 — Dashboard (Prism)
06.9	Module 8 — Chaos Tests
07	Key Implementation Code Snippets
08	Nutanix Concept Mapping Table
09	Featurising for Hiring
10	Interview Preparation
11	Resources & Next Steps

01 — Introduction & Project Vision

NanoFabric is a miniature, self-healing distributed storage cluster you build on your own laptop. It replicates the core internals of Nutanix's Distributed Storage Fabric (DSF / AOS) — the same technology that makes Nutanix's HCI platform fault-tolerant and self-managing. When you can sit across from a Nutanix interviewer and say 'I rebuilt the core concepts of AOS, Curator, and Prism myself,' and then show a running demo where you kill a node and watch data heal — you will be remembered.

Core Promise: After 4 weeks, you will have a running cluster where killing a node causes automatic re-replication with zero data loss — visible in a browser dashboard. This single demo covers Chapters 6 & 7 of the Nutanix Bible in code.

Why This Project Works for Hiring

Signals Depth: Not a CRUD app — a real distributed systems project tackling consensus, replication, and fault-tolerance.

Demoable in 30 Seconds: Kill node → data heals. Recruiters and HMs remember it.

Unlocks Interview Loops: Every design decision becomes a 20-minute discussion where you set the agenda.

Maps to Real Product: You can say 'my Curator is like Nutanix's Curator' — and mean it precisely.

The Two-Track Strategy

Run both tracks in parallel for the strongest possible portfolio:

Track	What	Proves
A — NanoFabric (Primary)	Build a toy distributed block-storage cluster that replicates, survives failures, and self-heals — with a web dashboard.	Deep internals understanding. Engineering / SRE signal.
B — Nutanix CE Lab (Parallel)	Deploy and operate a real Nutanix Community Edition cluster; sit the NCA certification exam.	Can run the actual product. Solutions / Support / Ops signal.

02 — Prerequisites & Preparation

Before You Write a Single Line of Code

Spend 2–3 days on setup and foundational reading. This investment pays back every hour of the build month with fewer dead-ends and richer interview answers.

System Requirements

Requirement	Minimum	Recommended
OS	Windows 10 / macOS 12 / Ubuntu 22	Ubuntu 22 or macOS 14 (native containers)
RAM	8 GB	16 GB+ (for 5-node Docker cluster)
Disk	20 GB free	50 GB SSD free
Python	3.10	3.11+ (better async, better errors)
Docker	Docker Desktop 4.x	Docker Desktop + Compose v2
Node.js	18 LTS	20 LTS (for React dashboard)
Git	Any recent	Git 2.40+

Software to Install (Day 0 Checklist)

Tool	Get From	Purpose
Python 3.11+	python.org or pyenv	Core runtime
Docker + Docker Compose v2	docker.com/desktop	Simulates a real cluster on one machine
Node.js 20 LTS + npm	nodejs.org	React dashboard build toolchain
Git + GitHub account	git-scm.com	Version control + portfolio visibility
VS Code (recommended)	code.visualstudio.com	With Python, Docker, REST Client extensions
Postman or httpie	postman.com / httpie.io	Test your FastAPI endpoints manually

Foundational Reading (Complete Before Week 1)

- Nutanix Bible (nutanixbible.com) — Chapters on DSF, the write path (oplog → extent store), Curator, and Prism. Read these before starting code.
- The Raft Paper — 'In Search of an Understandable Consensus Algorithm' (raft.github.io). Skim it once now, read again after Module 3.
- Visual Raft Explainer — thesecretlivesofdata.com/raft — 10 minutes, animated, essential mental model.
- FastAPI Docs — fastapi.tiangolo.com — Getting Started and Concurrency sections (2 hours).
- Docker Getting Started — docs.docker.com/get-started — If Docker is new to you, do the first 4 parts.
- MIT 6.5840 Lab 2 description — Even just reading the spec gives you the mental model for Raft implementation.

Python Knowledge Needed: [async/await](#), [classes](#), [file I/O](#), [HTTP basics](#). You do NOT need to know distributed systems already — that's what this project teaches you.

03 — Tech Stack Deep Dive

Every tool below earns its place. Here's what it is, why you're using it, and what Nutanix concept it maps to.

Tool	What It Is	Role in NanoFabric	Nutanix Concept
Python 3.11+	High-level language with excellent async support.	Core language for all node logic, write paths, metadata, self-healing.	AOS is C++/Go in production — concepts transfer 100%.
FastAPI	Modern async Python web framework. Auto-generates OpenAPI docs. Fast as NodeJS.	Node-to-node HTTP API: replica shipping, heartbeats, metadata queries, client write/read.	East-west network between CVMs (Chapter 5.3 of Nutanix Bible).
uvicorn	ASGI server — runs FastAPI. Non-blocking, production-quality.	Start each node's HTTP server. Handles concurrent requests from other nodes.	The network stack each CVM runs.
Docker + Compose v2	Containers + multi-container orchestration.	Each node = one container. docker compose up --scale node=5 gives you a real 5-node cluster.	Physical nodes in a Nutanix cluster. Killing a container = killing a node.
Local file system (append-only log)	Files + Python file I/O.	The oplog (write journal) per node. Writes land here first before being compacted to the extent store.	Nutanix oplog — the write journal in each CVM's storage engine.
SQLite (per-node)	Embedded SQL DB; zero deps.	The extent store: permanent block storage after oplog flush. Also used for node-local metadata cache.	Extent store — permanent on-disk block storage.
Python dict (in-memory)	RAM dictionary.	Read cache tier in front of disk. Cache hits avoid disk reads.	Unified Cache / Content Cache — RAM→SSD→HDD tiering.
Metadata Service (custom)	A replicated key-value map + gossip protocol you build.	Maps block_id → {copies: [node_ids], version: n}. Replicated across nodes for resilience.	Cassandra / Medusa — Nutanix's distributed metadata store.
React 18 + Vite	UI library + blazing-fast dev server.	The browser dashboard: node health grid, block heatmap, live write animation, Kill/Add buttons.	Prism Element — the management console.
Recharts	Composable chart library for React.	Live graphs: IOPS, latency, under-replicated block count over time.	Prism monitoring charts.

Tool	What It Is	Role in NanoFabric	Nutanix Concept
pytest + pytest-asyncio	Python testing framework with async support.	Unit tests + chaos tests: kill node mid-write, partition cluster, assert zero data loss.	X-Ray — Nutanix's failure-injection benchmarking tool.
httpx (async HTTP client)	Async-first HTTP client for Python.	Used inside node code to ship replicas and send heartbeats to other nodes asynchronously.	The inter-CVM network calls.

Stretch / Level-Up Swaps (Optional)

Replace FastAPI with gRPC + Protobuf: How production distributed systems actually talk. Nutanix uses this internally.

Rewrite the storage I/O engine in Go: Go is the language of Kubernetes, etcd, Docker. Signals serious systems engineering.

Replace hand-rolled metadata with real Raft (raft-cluster library): True consensus — the hardest and most-interviewed distributed systems topic.

Add Prometheus metrics + Grafana dashboard: Production observability — shows you understand the operations side too.

Deploy on Kubernetes with Helm: Nutanix is heavily cloud-native; K8s fluency is a strong hiring signal.

[illegible]

```
nanofabric/ ■■■ README.md ← Your portfolio centerpiece ■■■ docker-compose.yml ← Spins up N
nodes + metadata + dashboard ■■■ nanofabric/ ■■■ node/ ■■■ storage_engine.py ← Module
1: oplog + extent store + RAM cache ■■■ node_server.py ← Module 2: FastAPI HTTP API +
heartbeats ■■■ replication.py ← Module 4: RF write path, replica shipping ■■■
metadata/ ■■■ metadata_service.py ← Module 3: block→node map, gossip/replication ■■■
cluster/ ■■■ membership.py ← Module 2: who's alive ■■■ quorum.py ← Module 5:
majority voting, partition handling ■■■ curator.py ← Module 6: self-healing /
re-protection ■■■ client/ ■■■ client.py ← A tiny 'VM' that writes/reads blocks ■■■
dashboard/ ← Module 7: React + Vite ■■■ src/ ■■■ App.jsx ■■■ components/ ■■■
■■■ api.js ■■■ package.json ■■■ tests/ ■■■ test_storage.py ■■■ test_replication.py ■■■
■■■ test_chaos.py ← Module 8: kill-a-node, partition, fill-disk ■■■ docs/ ■■■
architecture.md ■■■ life of a write.md ← Narrate it: your interview crib sheet
```


05 — 4-Week Roadmap

~1–2 focused hours per day. Each week ends with something working and demo-able. If you fall behind, protect Weeks 1–3 (the MVP). Week 4 is the polish that makes it shine.

WEEK 0

Setup & Foundations (2–3 Days)

- Install Python 3.11, Docker, Node.js, Git — verify each runs
- Create GitHub repo 'nanofabric' with README skeleton and folder structure
- Read Nutanix Bible: DSF, oplog, extent store, Curator, Prism sections
- Skim Raft paper and watch thesecretlivesofdata.com/raft animation
- Start Nutanix CE download (Track B) — it's large, do it now
- Run a FastAPI 'Hello World' in Docker — verify your toolchain works

Done When: Docker container runs a FastAPI endpoint. GitHub repo exists.

WEEK 1

Single Node + Cluster Membership

- Module 0: Full project scaffolding, Makefile, .env config
- Module 1: StorageEngine class — oplog (append-only file) + SQLite extent store + RAM cache dict
- Durability test: write 100 blocks, restart process, read all back correctly
- Module 2: FastAPI node server with /write, /read, /status endpoints
- Heartbeat system: nodes ping metadata service every second
- Failure detection: mark node DOWN after 3 missed heartbeats (~3 seconds)

Done When: 3 nodes start, heartbeat, one gets killed — others detect DOWN within 3s.

WEEK 2

Distributed Metadata + Replication

- Module 3: MetadataService — block_id → {copies: [node_ids], version: n}
- Gossip replication: metadata service state copied to all nodes for resilience
- Module 4: RF write path — local write first (data locality), then synchronous replica shipping
- RF2 and RF3 modes with toggle in config
- Write acknowledgement: client only gets OK after ALL RF copies confirmed
- Integration test: write a block, inspect 2–3 nodes, confirm all copies present

Done When: RF2/RF3 synchronous writes work. You can narrate 'the life of a write' and demo it.

WEEK 3**Quorum + Self-Healing (MVP Done)**

- Module 5: Quorum check before writes — majority of nodes must be reachable
- Network partition simulation: block traffic between node groups
- Minority group refuses writes; majority keeps serving — no contradictory data
- Module 6: Curator background task — scans metadata for under-replicated blocks
- Auto re-replication: surviving nodes copy under-replicated blocks to healthy peers
- Node rejoin: handle node coming back — metadata sync, drop excess copies
- Chaos test: RF2, write data, kill a node, assert all blocks back to RF2 copies

Done When: Kill a node → every block re-replicates to healthy nodes → zero data loss. RECORD THIS.

WEEK 4**Dashboard + Polish + Presentation**

- Module 7: React + Vite dashboard — node health grid (green/red), block heatmap
- Live write animation showing data fanning to replica nodes
- Kill Node / Add Node buttons wired to actual cluster API
- Recharts: live IOPS, latency, under-replicated count graphs
- Module 8: pytest chaos suite — kill mid-write, partition, fill-disk assertions
- Docker Compose: docker compose up brings up full 5-node cluster + dashboard
- Record 60–90s demo video/GIF (the single highest-leverage deliverable)
- Write docs/life_of_a_write.md — your interview crib sheet
- Polish README: architecture diagram, concept-mapping table, demo GIF at top
- Track B: Sit the NCA certification exam

Done When: Anyone can run 'docker compose up' and see your full working cluster with dashboard.

06 — Module-by-Module Build Guide

MVP = Modules 0–6. Completing these gives you a CLI-driven, 3-node, RF=3, self-healing KV store — a genuinely impressive project on its own. Modules 7–8 make it recruiter-stopping.

Module 0 — Project Scaffolding

Goal: A clean, runnable project structure that builds and tests.

Steps:

Create the repo:

```
$ mkdir nanofabric && cd nanofabric $ git init && git remote add origin
https://github.com/YOU/nanofabric $ python3 -m venv venv && source venv/bin/activate
```

Install core dependencies:

```
$ pip install fastapi uvicorn httpx pytest pytest-asyncio $ pip install 'python-dotenv'
pydantic $ pip freeze > requirements.txt
```

Create the folder structure:

```
$ mkdir -p nanofabric/{node,metadata,cluster,client} $ mkdir -p tests docs dashboard $
touch nanofabric/__init__.py nanofabric/node/__init__.py
```

Create .env config:

```
NODE_ID=node1 NODE_HOST=0.0.0.0 NODE_PORT=8001 METADATA_URL=http://metadata:8000 RF=3 #
replication factor DATA_DIR=./data
```

Done When: python -m pytest tests/ runs (even with one trivial test that passes).

Module 1 — Local Storage Engine

Goal: A single node can durably store and retrieve blocks. Data survives process restart.

The Two-Layer Write Path:

Writes land in the **oplog** first (an append-only file — fast, sequential). A background thread flushes committed entries to the **extent store** (SQLite — permanent, queryable). Reads check the RAM cache, then extent store, then oplog.

```
# nanofabric/node/storage_engine.py
import sqlite3, json, os, threading
from pathlib import Path

class StorageEngine:
    def __init__(self, data_dir: str, node_id: str):
        self.data_dir = Path(data_dir)
        self.data_dir.mkdir(parents=True, exist_ok=True)
        self.node_id = node_id
        self._cache: dict = {} # RAM read cache (in-memory tier)
        self._lock = threading.Lock()
        self._init_oplog()
        self._init_extent_store()
        self._replay_oplog() # recover from crash
        def _init_oplog(self):
            self.oplog_path = self.data_dir / "oplog.jsonl"
            self.oplog = open(self.oplog_path, "a", buffering=1) # line-buffered
        def _init_extent_store(self):
            self.db = sqlite3.connect( str(self.data_dir / "extent_store.db"), check_same_thread=False)
            self.db.execute( "CREATE TABLE IF NOT EXISTS blocks " "(block_id TEXT PRIMARY KEY, data BLOB, version INTEGER)" )
            self.db.commit()
        def _replay_oplog(self):
            """On startup, replay the oplog to rebuild in-memory state."""
            if not self.oplog_path.exists():
                return
            with open(self.oplog_path) as f:
                for line in f:
                    entry = json.loads(line.strip())
                    if entry["op"] == "write":
                        self._apply_to_extent(entry["block_id"], entry["data"], entry["version"])
            def _apply_to_extent(self, block_id, data, version):
                self.db.execute( "INSERT OR REPLACE INTO blocks VALUES (?, ?, ?)", (block_id, data, version) )
                self.db.commit()
                self._cache[block_id] = data # warm the cache
            def write(self, block_id: str, data: str, version: int = 1) -> bool:
                with self._lock:
                    # Step 1: Write to oplog first (durability guarantee)
                    entry = {"op": "write", "block_id": block_id, "data": data, "version": version}
                    self.oplog.write(json.dumps(entry) + "\n")
                    self.oplog.flush()
                    os.fsync(self.oplog.fileno()) # force to disk
                    # Step 2: Apply to extent store + cache
                    self._apply_to_extent(block_id, data, version)
                return True
            def read(self, block_id: str) -> str | None:
                # Tier 1: RAM cache
                if block_id in self._cache:
                    return self._cache[block_id]
                # Tier 2: Extent store
                row = self.db.execute( "SELECT data FROM blocks WHERE block_id=?", (block_id,) ).fetchone()
                if row:
                    self._cache[block_id] = row[0]
                    # promote to cache
                    return row[0]
                return None
            def delete(self, block_id: str) -> bool:
                with self._lock:
                    self._cache.pop(block_id, None)
                    self.db.execute("DELETE FROM blocks WHERE block_id=?", (block_id,))
                    self.db.commit()
                return True
            def list_blocks(self) -> list[str]:
                rows = self.db.execute("SELECT block_id FROM blocks").fetchall()
                return [r[0] for r in rows]
```

Done When: Write 100 blocks, stop the process (kill -9), restart it, read all 100 blocks back correctly. Durability confirmed.

Module 2 — Cluster Membership & Heartbeats

Goal: Multiple nodes form a cluster. Each advertises liveness; dead nodes are automatically detected within ~3 seconds.

```
# nanofabric/node/node_server.py (key excerpts)
from fastapi import FastAPI
import httpx, asyncio, os, time
from .storage_engine import StorageEngine

app = FastAPI()
NODE_ID = os.getenv("NODE_ID", "node1")
METADATA_URL = os.getenv("METADATA_URL", "http://metadata:8000")
engine = StorageEngine(data_dir=f"./data/{NODE_ID}", node_id=NODE_ID)

@app.on_event("startup")
async def startup():
    # Register with the metadata service
    async with httpx.AsyncClient() as c:
        await c.post(f"{METADATA_URL}/nodes/register", json={"node_id": NODE_ID, "address": f"http://{NODE_ID}:{os.getenv('PORT', '8001')}"})
    # Start heartbeat background task
    asyncio.create_task(heartbeat_loop())

    async def heartbeat_loop():
        """Send heartbeat to metadata service every 1 second."""
        while True:
            try:
                async with httpx.AsyncClient(timeout=2.0) as c:
                    await c.post(f"{METADATA_URL}/nodes/heartbeat", json={"node_id": NODE_ID, "ts": time.time()})
            except Exception:
                pass
            # if metadata unreachable, keep trying
            await asyncio.sleep(1.0)

    @app.get("/status")
    async def status():
        return {"node_id": NODE_ID, "blocks": len(engine.list_blocks()), "alive": True}

    @app.post("/write")
    async def write_local(block_id: str, data: str, version: int = 1):
        """Called by the primary node to write a replica to this node."""
        ok = engine.write(block_id, data, version)
        return {"ok": ok}

    @app.get("/read/{block_id}")
    async def read_local(block_id: str):
        data = engine.read(block_id)
        if data is None:
            return {"found": False}
        return {"found": True, "data": data}
```

Done When: Start 3 nodes, kill one container, confirm the metadata service marks it DOWN within ~3 seconds (3 missed heartbeats × 1s interval).

Module 3 — Distributed Metadata Service

Goal: A metadata service tracks which nodes hold copies of each block. It knows which nodes are alive. Its state is replicated so it's not a single point of failure.

```
# nanofabric/metadata/metadata_service.py (key excerpts)
from fastapi import FastAPI
import time, threading
app = FastAPI()
HEARTBEAT_TIMEOUT = 3.0 # seconds
# In-memory cluster state (replicated to nodes via gossip in Module 3b)
nodes: dict = {} # node_id -> {address, last_seen, status}
block_map: dict = {} # block_id -> {copies: [node_ids], version: int}
_lock = threading.Lock()

@app.post("/nodes/register")
async def register(node_id: str, address: str):
    with _lock:
        nodes[node_id] = {"address": address, "last_seen": time.time(), "status": "UP"}
    return {"registered": node_id}

@app.post("/nodes/heartbeat")
async def heartbeat(node_id: str, ts: float):
    with _lock:
        if node_id in nodes:
            nodes[node_id]["last_seen"] = ts
            nodes[node_id]["status"] = "UP"
    return {"ok": True}

@app.get("/nodes")
async def get_nodes():
    now = time.time()
    with _lock:
        for nid, info in nodes.items():
            if now - info["last_seen"] > HEARTBEAT_TIMEOUT:
                info["status"] = "DOWN"
    return nodes

@app.post("/blocks/record")
async def record_block(block_id: str, copies: list[str], version: int):
    with _lock:
        block_map[block_id] = {"copies": copies, "version": version}
    return {"recorded": block_id}

@app.get("/blocks/{block_id}")
async def get_block(block_id: str):
    return block_map.get(block_id, {})

@app.get("/blocks/under-replicated")
async def under_replicated(rf: int = 3):
    """Return blocks with fewer than RF healthy copies – the Curator's input."""
    alive = {nid for nid, info in nodes.items() if info["status"] == "UP"}
    result = {}
    for bid, info in block_map.items():
        healthy_copies = [n for n in info["copies"] if n in alive]
        if len(healthy_copies) < rf:
            result[bid] = {"healthy_copies": healthy_copies, "needed": rf - len(healthy_copies)}
    return result
```

Done When: Write a block, query `/blocks/{block_id}`, see exactly which node IDs hold copies. Query `/nodes`, see health status with accurate UP/DOWN.

Module 4 — Replication Factor & The Life of a Write

This is THE star module. The entire 'life of a write' narrative — the single most important interview answer — is implemented here.

Goal: Every write is stored on RF distinct nodes synchronously. The client only receives OK after ALL copies are confirmed.

```
# nanofabric/node/replication.py – The Life of a Write
import httpx, asyncio from
.storage_engine import StorageEngine
async def replicated_write( block_id: str, data:
str, local_engine: StorageEngine, metadata_url: str, node_id: str, rf: int = 3, ) ->
dict: """ The complete write path – mirrors Nutanix AOS exactly: Step 1: Write to local
oplog (data locality – I/O served from local CVM) Step 2: Choose RF-1 other healthy
nodes (balanced placement) Step 3: Ship copies to each replica node SYNCHRONOUSLY Step
4: Only acknowledge OK after ALL RF copies are confirmed Step 5: Record placement in
metadata service """ # Step 1: Data locality – write to THIS node first version = 1 # use
a real clock/lamport timestamp in production local_engine.write(block_id, data, version)
# Step 2: Pick RF-1 other healthy nodes async with httpx.AsyncClient() as c: resp =
await c.get(f"{metadata_url}/nodes") all_nodes = resp.json() alive_others = [ (nid,
info["address"]) for nid, info in all_nodes.items() if info["status"] == "UP" and nid !=
node_id ] if len(alive_others) < rf - 1: # Not enough nodes – could raise or write with
degraded RF raise RuntimeError( f"Only {len(alive_others)} other nodes alive; need
{rf-1} replicas") # Pick by load (simplified: random; in prod use consistent hashing)
import random chosen = random.sample(alive_others, rf - 1) # Step 3: Ship replicas
SYNCHRONOUSLY – ALL must succeed async def ship(nid: str, address: str) -> bool: try:
async with httpx.AsyncClient(timeout=5.0) as c: r = await c.post(f"{address}/write",
params={"block_id": block_id, "data": data, "version": version}) return r.status_code ==
200 and r.json().get("ok") except Exception: return False results = await
asyncio.gather(*[ship(nid, addr) for nid, addr in chosen]) if not all(results): # Step
4: FAIL – not all copies acknowledged raise RuntimeError("Replica write failed – not all
copies acknowledged") # Step 4: All copies confirmed – now record in metadata all_copies
= [node_id] + [nid for nid, _ in chosen] async with httpx.AsyncClient() as c: await
c.post(f"{metadata_url}/blocks/record", params={"block_id": block_id, "version":
version}, json=all_copies) # Step 5: Return success to client return {"ok": True,
"block_id": block_id, "copies": all_copies, "rf": rf}
```

Done When: Write a block with RF=2. Inspect both node containers' extent_store.db — confirm the block exists in both. Confirm the client waited for both copies before returning OK.

Module 5 — Quorum & Split-Brain Prevention

Goal: The cluster only accepts writes when a majority of nodes (quorum) are reachable. A partitioned minority group must refuse writes to prevent divergent data (split-brain).

```
# nanofabric/cluster/quorum.py import httpx async def check_quorum(metadata_url: str,
required_ratio: float = 0.5) -> bool: """ Returns True if > 50% of registered nodes are
UP. Write operations MUST call this first. This is why Nutanix (and Raft) require an odd
number of nodes (3, 5, 7): you always get a clear majority. """ async with
httpx.AsyncClient(timeout=2.0) as c: resp = await c.get(f"{metadata_url}/nodes") nodes =
resp.json() total = len(nodes) alive = sum(1 for n in nodes.values() if n["status"] ==
"UP") if total == 0: return False return (alive / total) > required_ratio # strict
majority # In node_server.py, wrap writes: @app.post("/client/write") async def
client_write(block_id: str, data: str): from nanofabric.cluster.quorum import
check_quorum if not await check_quorum(METADATA_URL): return {"ok": False, "error": "No
quorum - cluster is partitioned or degraded"} result = await replicated_write(block_id,
data, engine, METADATA_URL, NODE_ID, rf=RF) return result
```

Done When: 3-node cluster, partition into groups of 2 and 1. Group of 2 keeps accepting writes. Group of 1 returns error 'No quorum'. No contradictory data exists after partition heals.

Module 6 — Self-Healing / Re-Protection (The Curator)

This is your headline feature. This is what wins interviews. When you can demo 'kill a node, watch it heal' live — you've demonstrated the core value proposition of Nutanix HCI in code.

Goal: When a node dies, a background Curator task detects under-replicated blocks and automatically re-replicates them to healthy nodes. RF is fully restored with zero data loss.


```
# nanofabric/cluster/curator.py – Your Nutanix Curator
import httpx, asyncio, logging
logger = logging.getLogger("curator")
async def curator_loop(metadata_url: str, rf: int = 3, interval: float = 5.0):
    """ Background self-healing task. Runs on the 'leader' node continuously.
    Every interval seconds: 1. Ask metadata service for under-replicated blocks
    2. For each, find a healthy node that has a copy 3. Ship a copy to a healthy node that DOESN'T have one
    4. Update metadata – RF restored This is exactly what Nutanix's Curator MapReduce service does. """
    while True:
        try:
            await _heal_under_replicated(metadata_url, rf)
        except Exception as e:
            logger.error(f"Curator error: {e}")
        await asyncio.sleep(interval)
    async def _heal_under_replicated(metadata_url: str, rf: int):
        async with httpx.AsyncClient(timeout=5.0) as c:
            # Get under-replicated blocks
            resp = await c.get(f"{metadata_url}/blocks/under-replicated", params={"rf": rf})
            under_rep = resp.json()
            # {block_id: {healthy_copies, needed}}
            if not under_rep:
                return # cluster is healthy
            # Get all alive nodes
            nodes_resp = await c.get(f"{metadata_url}/nodes")
            all_nodes = nodes_resp.json()
            alive_nodes = {nid: info for nid, info in all_nodes.items() if info["status"] == "UP"}
            for block_id, info in under_rep.items():
                healthy_copies = info["healthy_copies"]
                needed = info["needed"]
                # Find nodes that DON'T have this block and are alive
                candidates = [nid for nid in alive_nodes if nid not in healthy_copies]
                if not candidates or not healthy_copies:
                    logger.warning(f"Cannot heal {block_id}: no candidates or no source")
                    continue
                # Fetch data from a healthy copy
                source_id = healthy_copies[0]
                source_addr = all_nodes[source_id]["address"]
                data_resp = await c.get(f"{source_addr}/read/{block_id}")
                if not data_resp.json().get("found"):
                    logger.warning(f"Source {source_id} cannot serve {block_id}")
                    continue
                block_data = data_resp.json()["data"]
                # Ship to 'needed' candidates
                for target_id in candidates[:needed]:
                    target_addr = alive_nodes[target_id]["address"]
                    wr = await c.post(f"{target_addr}/write", params={"block_id": block_id, "data": block_data, "version": 1})
                    if wr.status_code == 200:
                        # Update metadata
                        healthy_copies.append(target_id)
                await c.post(f"{metadata_url}/blocks/record", params={"block_id": block_id, "version": 1, "json": healthy_copies})
            logger.info(f"Healed {block_id} → {target_id}")
```

Done When: RF=3, write 50 blocks, kill one node container. Within 10 seconds, query /blocks/under-replicated — it returns empty. Every block is back to 3 copies. A GET on any block_id still returns data. **Record this as a video immediately.**

Module 7 — The Dashboard ('Prism')

Goal: A React browser dashboard that visualizes the cluster live and lets you trigger kills and adds with a button click. This is your demo video subject.

Feature	Description
Node Health Grid	Green/red cards per node, showing heartbeat status, block count, last-seen time.
Block Placement Heatmap	Grid showing which blocks live on which nodes — updates in real-time.
Live Write Animation	When a write is triggered, animate it fanning out from primary to replicas.
RF Slider	Change replication factor from 2 to 5 — cluster adapts automatically.
Kill Node / Add Node Buttons	One click kills a container (triggers self-healing). One click brings it back.
Under-Replicated Count Chart	Recharts line graph: spikes when node dies, drops back to zero as Curator heals.
IOPS & Latency Meters	Live write/read throughput and p50 latency.

Key React snippet — polling node status:

```
// dashboard/src/api.js
const API = "http://localhost:8000"; // metadata service gateway
export const fetchNodes = () => fetch(`${API}/nodes`).then(r => r.json());
export const killNode = (nodeId) => fetch(`${API}/admin/kill/${nodeId}`, {method: "POST"}).then(r => r.json());

// dashboard/src/components/NodeGrid.jsx
import { useEffect, useState } from "react";
import { fetchNodes } from "../api";
export default function NodeGrid() {
  const [nodes, setNodes] = useState({});
  useEffect(() => {
    const id = setInterval(() => fetchNodes().then(setNodes), 1000);
    return () => clearInterval(id);
  }, []);
  return (
    Object.entries(nodes).map(([id, info]) => (
      {id} Status: {info.status} Blocks: {info.block_count ?? "-"}
    ))
  );
}
```

Module 8 — Chaos Tests

Goal: Automated tests that prove your cluster handles failures correctly. These are the tests that make hiring managers say 'this person thinks like an SRE.'

```
# tests/test_chaos.py import pytest, httpx, asyncio, time BASE = "http://localhost:8000"
# metadata / gateway @pytest.mark.asyncio async def test_no_data_loss_on_node_death():
    """Write 20 blocks with RF=2, kill one node, assert all blocks still readable.""" async
    with httpx.AsyncClient(base_url=BASE, timeout=10) as c: # Write 20 blocks block_ids =
        [f"chaos-block-{i}" for i in range(20)] for bid in block_ids: r = await
        c.post("/client/write", params={"block_id": bid, "data": f"value-{bid}"}) assert
        r.json()["ok"], f"Write failed for {bid}" # Kill node1 (the first replica node) await
        c.post("/admin/kill/node1") # Wait for Curator to heal (up to 30 seconds) for _ in
        range(30): await asyncio.sleep(1) under = await c.get("/blocks/under-replicated",
        params={"rf": 2}) if not under.json(): break # healed! assert not under.json(), "Curator
        did not restore RF=2 within 30 seconds" # Confirm all blocks are still readable for bid
        in block_ids: r = await c.get(f"/client/read/{bid}") assert r.json()["found"], f"Block
        {bid} lost after node death!" assert r.json()["data"] == f"value-{bid}", "Data
        corruption detected!" @pytest.mark.asyncio async def test_no_split_brain_on_partition():
    """Partition cluster 2-vs-1; minority must reject writes.""" async with
    httpx.AsyncClient(base_url=BASE, timeout=10) as c: await c.post("/admin/partition",
    json={"group_a": ["node1", "node2"], "group_b": ["node3"]}) await asyncio.sleep(2) #
    node3 (minority) should refuse writes r = await c.post("/client/write",
    params={"block_id": "split-test", "data": "oops"}, headers={"X-Target-Node": "node3"})
    assert not r.json().get("ok"), "Minority node accepted write – split-brain!" await
    c.post("/admin/heal-partition")
```

08 — Nutanix Concept Mapping Table

Put this table in your README and memorise it. Being able to map every component to its Nutanix equivalent proves you understand the real product, not just your toy.

NanoFabric Component	Nutanix Equivalent	What It Does
StorageEngine oplog	AOS oplog / Write journal	Append-only write journal — fast, sequential, durable writes
StorageEngine extent_store (SQLite)	AOS Extent Store	Permanent block storage on disk; data survives reboots
RAM cache dict	Unified / Content Cache	In-memory read tier; RAM → SSD → HDD tiering
node_server.py FastAPI	Stargate (I/O manager)	Handles all client and inter-node I/O; the write/read path
replication.py RF write path	AOS RF2/RF3 replication	Synchronous replication to N nodes; ack only after all confirm
metadata_service.py	Cassandra / Medusa	Distributed metadata: block → node placement index
membership.py heartbeats	Cluster health monitoring	Detects node failures via missed heartbeats
quorum.py	Zeus / Zookeeper	Cluster config, leader election, quorum enforcement
curator.py background loop	Curator (self-healing)	Re-replicates under-protected data after node failures
Docker container per node	Physical node + CVM	Each container = one converged compute+storage node
Docker Compose cluster	Nutanix cluster of nodes	N nodes forming one shared storage fabric
React dashboard	Prism Element	Web UI for cluster monitoring and operations
Recharts graphs	Prism monitoring charts	Live IOPS, latency, capacity, health visualizations
pytest chaos tests	X-Ray failure injection	Automated failure scenarios validating durability and healing

09 — Featurising for Hiring

The code is half the work. This section is what turns a good project into a hiring magnet. None of these take more than a few hours — but they 10x the impact.

1. The 60–90 Second Demo Video

Record your terminal or dashboard: cluster running → click 'Kill Node' → it goes red → under-replicated count spikes → self-healing restores it → a read of any block still returns data. Loop it as a GIF. Embed it at the very top of your README. This is the single highest-leverage thing you can create.

2. README with Concept-Mapping Table

Your README is your resume bullet made visible. Include: one-line pitch, architecture diagram, the concept-mapping table from Section 8, a 'Quick Start' section (docker compose up → working cluster in 30 seconds), and links to the blog post and demo video. A recruiter should understand the project in 90 seconds.

3. docs/life_of_a_write.md

Write the write path in plain English, step by step, as if explaining to a senior Nutanix engineer. Include your code's function names at each step. This document is your interview prep — practice saying it out loud until it's completely smooth. It will be asked.

4. A Blog Post or LinkedIn Article

'I built a miniature Nutanix to understand HCI internals.' Walk through your architecture decisions, the self-healing demo, and what you learned. 800–1,200 words. Post to dev.to or LinkedIn. Nutanix engineers and hiring managers genuinely engage with this. It generates inbound interest.

5. Resume Bullets

Example bullets (rewrite in your voice): 'Built NanoFabric, a distributed storage cluster implementing RF2/RF3 synchronous replication, quorum-based split-brain prevention, and automatic self-healing — surviving simulated node failures with zero data loss (Python, FastAPI, Docker, React).' And: 'Implemented a Curator-style background re-replication service; validated by chaos tests asserting zero data loss on node death.'

10 — Interview Preparation

Every module you build unlocks real answers to real interview questions. Here are the questions this project directly prepares you for:

Q: Walk me through the life of a write in your system.

A: Module 4 — data locality → choose replicas → synchronous replication → ack after all copies → record in metadata. Map each step to AOS: Stargate, oplog, extent store, Cassandra.

Q: How does your system detect and handle node failures?

A: Module 2 + 6 — heartbeats every 1s, DOWN after 3 missed. Then Curator scans metadata for under-replicated blocks and re-replicates to healthy nodes.

Q: What's quorum and why does it matter?

A: Module 5 — majority-node availability required before writes. Prevents split-brain where two partitioned groups write conflicting data. Why 3+ nodes: guaranteed majority always exists.

Q: Explain your consistency model. What CAP tradeoffs did you make?

A: CP system — choose consistency over availability. Writes fail if quorum can't be reached. Data is never lost or corrupted, but the cluster may become write-unavailable during partitions.

Q: How is data placed across nodes? Why not a simple hash?

A: Random balanced selection from healthy nodes (upgradeable to consistent hashing). Consistent hashing minimizes data movement when nodes join/leave — describe the ring.

Q: What breaks first at scale? How would you improve this?

A: The metadata service becomes a bottleneck at millions of blocks. Real solution: Nutanix shards metadata across Cassandra nodes with Paxos for consistency. I'd replace my dict with an etcd or sharded Redis.

Q: How did you test failure scenarios?

A: Module 8 — pytest chaos tests: kill node mid-write, assert zero data loss; partition cluster 2-vs-1, assert minority refuses writes; fill a disk, assert rebalancing kicks in.

Q: What's the difference between your Curator and Nutanix's real Curator?

A: Same job: scan for under-replicated data, trigger re-replication. Real Curator is a distributed MapReduce service running across CVMs, handles erasure coding, dedup, compression, disk balancing. Mine is a single background loop — same concept, simplified implementation.

Prep Tip: Prepare a 2-minute and a 10-minute version of 'tell me about NanoFabric.' The 2-minute version covers: what it does, the self-healing demo, and one key design decision. The 10-minute version walks through the full architecture and every Nutanix mapping. Practice both out loud until they feel completely natural.

11 — Resources & Next Steps

ESSENTIAL — Read These First

Resource	Where	Why It Matters
Nutanix Bible	nutanixbible.com	The canonical free deep-dive on Nutanix architecture. Read the DSF, oplog, Curator, and Prism sections before you start coding.
The Raft Paper	raft.github.io	'In Search of an Understandable Consensus Algorithm.' Skim first, read deeply after Module 3. Prime interview material.
Visual Raft	theseترلivesofdata.com/raft	10-minute animated explainer. Essential mental model for leader election and log replication.
Designing Data-Intensive Applications	DDIA by Martin Kleppmann	The field bible. Replication, partitioning, and consistency chapters map 1:1 to your design decisions.

CORE BUILD REFERENCES

Resource	Where	Why It Matters
FastAPI Docs	fastapi.tiangolo.com	Complete framework reference. Focus on async endpoints, background tasks, and Pydantic models.
Docker Compose v2	docs.docker.com/compose	Especially: networks, health checks, and scaling (--scale flag for multi-node clusters).
httpx (async HTTP)	www.python-httpx.org	Your inter-node communication library. Async client, timeouts, error handling.
pytest-asyncio	pytest-asyncio.readthedocs.io	For testing async FastAPI code. Essential for your chaos tests.

DISTRIBUTED SYSTEMS DEPTH

Resource	Where	Why It Matters
MIT 6.5840 / 6.824	pdos.csail.mit.edu/6.824	Labs literally build Raft and a replicated KV store. Even reading the lab specs gives deep mental models.
raft-cluster (PyPI)	pypi.org/project/raft-cluster	Python Raft implementation if you want to upgrade from hand-rolled metadata to true consensus.
CAP Theorem Explained	search 'CAP theorem Martin Kleppmann'	Understanding CP vs AP is essential for interview discussions about your consistency choices.

DASHBOARD & TOOLING

Resource	Where	Why It Matters
React + Vite	vitejs.dev	Fastest React dev setup. Create with: <code>npm create vite@latest dashboard -- --template react</code>
Recharts	recharts.org	Composable charts for React. Use LineChart for the under-replicated count time series.
Tailwind CSS	tailwindcss.com	Utility-first CSS. Fastest way to make your dashboard look professional.

Your First Three Actions — Do These Today

- 1 Create the GitHub repo 'nanofabric' with README skeleton and the folder structure from Section 4.
- 2 Run `pip install fastapi uvicorn httpx pytest` and confirm a FastAPI 'Hello World' runs in Docker.
- 3 Read the Nutanix Bible sections on DSF, the oplog → extent store write path, and Curator. This reading shapes everything.

By the time NanoFabric heals itself on screen, you won't just know Nutanix — you'll have rebuilt its beating heart. That's what gets you hired.